实验一：C-- 词法分析器与语法分析器的实现

1. 实验功能与实现方法

本实验实现了词法分析和语法分析，能够输出词法错误和语法错误，如果没有错误则输出语法树。

1.1 词法分析

词法分析部分由 lexical.l 实现。Flex
根据正则表达式识别关键字、标识符、整数、浮点数、运算符和分隔符，并将识别出的 token 返回给 Bison。每个合法 token 同时生成一个语法树叶结点，通过 yylval 传递给语法分析器。

程序能够识别实验要求中的基本词法单元，并仅实现选做 1.1：八进制和十六进制整数。

八进制和十六进制整数：例如 0123 和 0x3F，输出时分别转换为十进制值 83 和 63；对于 09、0x3G 等非法形式，直接报告 Type A 词法错误。

对于未定义字符，如 ~，程序输出：

```
Error type A at Line 4: Mysterious character "~".
```

1.2 语法分析与语法树

语法分析部分由 syntax.y 实现，文法按照附录 A 的 C-- 文法编写。当前支持的主要语法结构包括：

全局变量、局部变量及局部初始化；
一维和多维数组、数组访问；
struct 定义、结构体类型引用和成员访问；
无参/有参函数定义与函数调用；
赋值、算术、关系和逻辑表达式；
return、if、if-else、while 以及嵌套复合语句。

Bison 在每次归约时利用语义动作构造语法树。终结符结点由 Flex
创建，非终结符结点则在产生式归约时通过 $1、$2、$3 等子结点组合得到 $$。解析完成后从 Program 根结点递归打印语法树。空产生式不生成结点，因此输出格式与实验样例保持一致。

对于表达式歧义，程序显式设置了运算符优先级和结合性，例如乘除高于加减、赋值为右结合，并对数组下标、结构体成员访问和 dangling else 进行了优先级处理，从而消除了 Bison 的 shift/reduce 冲突。

1.3 语法错误处理

对于语法错误，程序输出 Type B 错误，并记录出错行号。除 Bison 的基本错误报告外，还加入了错误恢复规则，使分析器在遇到部分错误后能够继续向后分析，而不是在第一个错误处直接终止。

2. 编译与运行方法

实验目录中提供了 Makefile。需要系统中已安装：

gcc
flex
bison

本实验实际开发与测试环境为 WSL2，GCC 13.3.0、GNU Flex 2.6.4、GNU Bison 3.8.2；程序未使用额外第三方函数库。

在 Ubuntu / WSL 环境下进入源代码目录后执行：

```
cd Code
make clean
make
```

make 的主要过程依次为：

```
bison -d -v -o syntax.tab.c syntax.y
flex -o lex.yy.c lexical.l
gcc -std=c99 -D_POSIX_C_SOURCE=200809L -c syntax.tab.c -o syntax.tab.o
gcc -o parser syntax.tab.o -lfl
```

其中 Bison 必须先运行，以生成 syntax.tab.h

编译完成后生成可执行文件 parser。运行方式为：

```
./parser path/to/input.cmm
```

例如：

```
./parser test.cmm
```

如果输入无词法或语法错误，程序打印语法树；如果存在错误，则打印 Type A / Type B 错误信息。语法树和错误信息均输出到 stdout，处理完成后正常返回 0。

当前仅实现选做 1.1（八进制和十六进制整数）；指数形式浮点数和注释两个选做要求均未实现。